Hackathon Logistics - Timeline

- API keys and Autograder/Kaggle for the Early checkpoint will be released by EOD today
- Two graded deadlines: March 13 and April 7!

Date	Event	Description
02/26	Hackathon release	Dataset and problem statement are available.
03/13	Early checkpoint: Prediction list (required & graded)	Submit checkpoint predictions by 11:59 PM on this day. Achieving a Spearman Correlation of >0.02 will grant full credit for the checkpoint.
04/03	First leaderboard submission (optional, non-graded)	We will run the evaluation on all submissions that are submitted by 11:59 PM on this day and release the results to you. While this submission is optional and will not be graded, we strongly encourage you to submit your model's predictions to test the prediction performance of your developed model at this point.
04/06	Second leaderboard submission (optional, non-graded)	We will run the evaluation on all submissions that are submitted by 11:59 PM on this day and release the results to you. While this submission is optional and will not be graded, we strongly encourage you to submit your model's predictions to test the prediction performance of your developed model at this point.
04/07	Final submission: Prediction list (required & graded)	Submit final **predictions**. Submit your final prediction. We will refresh the leaderboard for the last time after the submission deadline and calculate your final score.
04/07	Final submission: Final report (required & graded)	Submit **the final report PDF** on Canvas

Submission details

Deadline 1 - Early Checkpoint (March 13)

On Gradescope:

- Spearman predictions (predictions.csv)
- Group number (GroupName.txt)
- API key (APIKey.txt)

Deadline 2 - Final Submission (April 7)

On Gradescope:

- Spearman predictions (predictions.csv)
- Top 10 mutants (top10.txt)
- Group number (GroupName.txt)
- API key (APIKey.txt)

On Canvas:

- Final report PDF

Hackathon Logistics – Grading breakdown

Category	Description	Weight
Early checkpoint	Each group needs to submit predictions by the checkpoint deadline. Achieving a Spearman Correlation of >0.02 will grant full credit for the checkpoint.	5%
Spearman Correlation	Measures ranking consistency between predicted and actual mutation effects in the test set	45%
Top 10 Mutations	Measures the quality of the top 10 mutation recommendations provided by each participant's model	15%
Final Report	Each group's final report describing their querying, training, and model architecture selection	35%
Leaderboard Bonus	Bonus points awarded to the top participants on the final Gradescope leaderboard	Up to 10%

Leadership Bonuses (on Gradescope)

- 1st place: +10% bonus points
- 2nd - 3th place: +5% bonus points
- 4th - 6th place: +3% bonus points

Spearman Correlation

- Achieving any Spearman Correlation >0.02 for the Early Checkpoint will give full credit (aka 5% of the overall hackathon grade!)
- Test set from test.csv

Example predictions

mutant	DMS_score_predicted
M0A	0.073
V1D	0.688
K59L	0.236

Spearman Correlation (Test Set)	Score Percentage
≥ 0.35	110% (Full Credit +10% Bonus)
0.29 - 0.35	100% (Full Credit)
0.26 - 0.29	90%
0.23 - 0.26	80%
0.2 - 0.23	70%
0.15 - 0.2	60%
0.1 - 0.15	50%
0.05 - 0.1	40%
0.02 - 0.05	30% (checkpoint)
< 0.02	0% (No Credit)

Top 10 Mutations

Top10.txt:

- List of 10 mutants, each on an individual line

Top Percentage Recalled	Points Awarded
1%	3 (Full Credit)
2%	2.5
5%	2
10%	1.5
20%	1
50%	0.5
100%	0 (No Credit)

Final Report

There is no page limit, but we expect around 2-4 pages for sufficient detail

Your report should include:

1) **Querying Strategy:** rationale behind your three queries
2) **Model Architecture and Justification:** rationale behind architecture choice, modifications, number of layers, activation functions, loss functions, etc.
3) **Training Details:** data preprocessing, feature engineering, hyperparameter tuning, optimization strategies
4) **Model Performance Analysis:** discuss any performance metrics used to evaluate your model's effectiveness
5) **Reflections and Challenges:** impact of different design choices, limitations of your approach, challenges you faced
6) **Code Submission:** github or cloud drive link and README file detailing how to run your code
7) **Contributions:** which team members worked on which parts

Hackathon Logistics – Suggested workflow

Step 1 - Getting Started for Checkpoint:

1. Download the skeleton code: We have provided some skeleton code that you can use to get started. Download it and familiarize yourself with the provided initial dataset. Read the background section in the project docs.
2. Set up your computing environment. The skeleton code uses Colab, but you may want ot familiarize yourself with PACE ICE as well.
3. Familiarize yourself with the provided skeleton code and begin to experiment with changes to it. Making changes to the skeleton code should be enough to yield a spearman correlation of >0.02.
4. Submit your checkpoint predictions.csv via Gradescope assignment "Checkpoint". Iterate until you achieve full credit for the Early Checkpoint.

```python
class ProteinDataset(Dataset):
    def __init__(self, df, istrain=True):

        alphabet = 'ACDEFGHIKLMNPQRSTVWY'
        map_a2i = {j:i for i,j in enumerate(alphabet)}
        map_i2a = {i:j for i,j in enumerate(alphabet)}

        self.df = df

        self.num_samples = len(self.df)
        self.seq_length = len(self.df.sequence.values[0])
        self.num_channels = 20

        # TODO: replace one-hot encodings with your own encodings
        self.encodings = np.zeros((self.num_samples, self.num_channels, self.seq_length)).astype(np.float32)
        self.targets = np.zeros(self.num_samples).astype(np.float32)

        if istrain:
          for it, (seq,target) in enumerate(self.df[['sequence', 'DMS_score']].values):
              for i,aa in enumerate(seq):
                  self.encodings[it,map_a2i[aa],i] = 1
              self.targets[it] = target

          self.encodings = self.encodings.astype(np.float32)
          self.targets = self.targets.astype(np.float32)
        else:
          for it, seq in enumerate(self.df['sequence'].values):
              for i,aa in enumerate(seq):
                  self.encodings[it,map_a2i[aa],i] = 1

          self.encodings = self.encodings.astype(np.float32)
```

Example script provides one-hot encoding + linear regression

```python
X_train, y_train = next(iter(train_loader))

regressor = LinearRegression()
regressor.fit(X_train.view(X_train.size(0), -1).numpy(), y_train.numpy())

X_test, _ = next(iter(test_loader))
y_test_pred = regressor.predict(X_test.view(X_test.size(0), -1).numpy())
```

Hackathon Logistics – Suggested workflow

Step 2 - Active Learning and Leaderboard Iteration Loop:

1. Design and optimize your model, train and validate on your training data. Optimize with hyperparameter tuning, cross-validation, initial performance assessment w/ Spearman correlation
2. Submit your checkpoint predictions to the Kaggle Leaderboard (subset of full test set) to evaluate your method
3. Submit your query of up to 100 mutations (ensure query.txt format correctness via Gradescope's format check).
4. Incorporate queried data into your training dataset once received.

```
# Example: randomly select 100 test variants to be queried.
# Note: random selection may not be a good strategy
# TODO: select query mutants for the next round based on your own criteria

querys = np.random.choice(df_test.mutant.values, size=100, replace=False)
querys
```

```
array(['K287E', 'D575G', 'F507R', 'T520I', 'R265E', 'E441M', 'E184M',
       'I562E', 'N648A', 'R344T', 'K413P', 'N474C', 'G161T', 'I137K',
       'N7K', 'V497I', 'F235E', 'E358V', 'G86P', 'F79P', 'I591H', 'L627S',
       'A174P', 'P370M', 'T652N', 'S392V', 'S392E', 'D584W', 'S493D',
       'W582D', 'M245H', 'W560N', 'S372T', 'Y375S', 'Q277D', 'G170N',
       'L264K', 'S483H', 'E307P', 'T328Y', 'N488D', 'V618R', 'K408W',
       'H95Q', 'E532W', 'Y610V', 'R376D', 'G498R', 'P564F', 'T572K',
       'L589Y', 'T450I', 'I607W', 'L264D', 'P96F', 'R45W', 'G565N',
       'E299W', 'D187I', 'E468K', 'K400S', 'G386C', 'Y537N', 'F507Y',
       'N536P', 'D575T', 'S271L', 'L424F', 'K371F', 'L539G', 'N629S',
       'G389P', 'S17C', 'R387D', 'V275G', 'N643E', 'M419Y', 'E398V',
       'P34G', 'A69S', 'G397Q', 'A352I', 'E184S', 'N2E', 'E567R', 'F223S',
       'D244V', 'S411A', 'I478N', 'L528Q', 'K145T', 'R5P', 'I481K',
       'I181M', 'M82P', 'C130S', 'F238Y', 'A510K', 'T450W', 'R343Q'],
      dtype=object)
```

Skeleton code randomly samples 100 variants to be queried

Some strategies to test out for sampling are (refer to the Supplementary info):

- Greedy Selection
- Upper confidence bound (UCB)
- Thompson Sampling
- Expected improvement (EI)
- MLDE (Machine learning directed evolution)
- Uncertainty based approaches

Hackathon Logistics – Suggested workflow

Step 3 - Final Submission:

1. Select your top 10 mutation predictions based on the final model.
2. Submit predictions.csv and top10.txt via Gradescope's leaderboard assignment once your model is finalized, make sure to check your format using the “FormatCheck” assignment
3. Prepare and finalize your detailed project report documenting your strategy, model architecture, training procedure, and results.
4. Submit the final project report.